# جزوه جامع و تخصصی سیستم‌عامل

## فصل ۵: زمان‌بندی پردازنده (CPU Scheduling)

### بخش اول: مفاهیم پایه و معیارها

### ۱. مقدمه و چرایی زمان‌بندی

در سیستم‌های تک‌هسته‌ای (Single-core)، در هر لحظه تنها **یک** پردازه (Process) امکان اجرا دارد. سایر پردازه‌ها باید منتظر بمانند تا CPU آزاد شود.

- **هدف چندبرنامگی (Multiprogramming):** ماکسیمم کردن بهره‌وری پردازنده (CPU Utilization).
- **روش کار:** هر زمان پردازه‌ای منتظر چیزی (مثلاً I/O) شد، سیستم‌عامل CPU را از او می‌گیرد و به پردازه دیگری می‌دهد تا زمان هدر نرود.

**نکته مدرن:** در سیستم‌عامل‌های امروزی، در واقع **نخ‌ها (Threads)** زمان‌بندی می‌شوند، نه کل پردازه. اما در مفاهیم کلی این فصل، واژه‌های "پردازه" و "نخ" معادل هم در نظر گرفته می‌شوند.

### ۲. چرخه انفجار پردازنده - ورودی/خروجی (CPU-I/O Burst Cycle)

موفقیت در زمان‌بندی وابسته به درک رفتار پردازه‌هاست. اجرای هر پردازه شامل تناوبی از دو حالت است:

1. **انفجار پردازنده (CPU Burst):** زمانی که پردازه مشغول محاسبات روی CPU است.
2. **انفجار I/O (I/O Burst):** زمانی که پردازه منتظر ورودی یا خروجی است.

- **روند اجرا:** (CPU Burst -> I/O Burst -> CPU Burst -> ... -> System Call (Terminate
- **توزیع انفجارها (Histogram):** تحقیقات نشان می‌دهد اکثر پردازه‌ها تعداد زیادی **انفجار CPU کوتاه** (کمتر از ۸ میلی‌ثانیه) و تعداد کمی **انفجار CPU طولانی** دارند (توزیع نمایی یا فوق‌نمایی).
  - **I/O-Bound:** برنامه‌هایی که بیشتر منتظر I/O هستند (انفجارهای CPU کوتاه و زیاد).
  - **CPU-Bound:** برنامه‌هایی که محاسباتی هستند (انفجارهای CPU طولانی و کم).

## ۳. زمان‌بند پردازنده (CPU Scheduler)

هرگاه پردازنده بیکار شود، سیستم‌عامل باید تصمیم بگیرد چه کسی بعدی است. این کار توسط **زمان‌بند کوتاه‌مدت (Short-term Scheduler)** انجام می‌شود.

- **وظیفه:** انتخاب یک پردازه از **صف آماده (Ready Queue)** و تخصیص CPU به آن.
- **ساختار صف آماده:** الزاماً یک صف FIFO نیست. می‌تواند صف اولویت‌دار (Priority Queue)، درخت یا لیست پیوندی نامرتب باشد.
- **محتوای صف:** معمولاً اشاره‌گرهایی به **PCB** (بلوک کنترل پردازه) ها در صف قرار می‌گیرند.

## ۴. زمان‌بندی پس‌گرفتنی در برابر غیرپس‌گرفتنی (Preemptive vs Non-preemptive)

**این بخش بسیار مهم و امتحانی است.** تصمیم‌گیری برای زمان‌بندی در ۴ شرایط رخ می‌دهد:

| شماره | وضعیت تغییر حالت | مثال | نوع زمان‌بندی مجاز |
|---|---|---|---|
| ۱ | اجرا به انتظار (Running to Waiting) | درخواست I/O یا دستور wait() | فقط غیرپس‌گرفتنی |
| ۲ | اجرا به آماده (Running to Ready) | وقوع وقفه (Interrupt) | **پس‌گرفتنی** |
| ۳ | انتظار به آماده (Waiting to Ready) | تکمیل شدن I/O | **پس‌گرفتنی** |
| ۴ | خاتمه (Terminate) | پایان برنامه | فقط غیرپس‌گرفتنی |

### *الف) زمان‌بندی غیرپس‌گرفتنی (Non-preemptive / Cooperative)*

- فقط در شرایط **۱ و ۴** رخ می‌دهد.
- **ویژگی:** وقتی CPU به پردازه داده شد، تا زمانی که خودش نخواهد (اتمام کار یا رفتن به I/O)، پردازنده را رها نمی‌کند.
- **کاربرد:** ویندوز ۳.۱ و مک‌های قدیمی. (امروزه کمتر استفاده می‌شود چون یک باگ در برنامه کل سیستم را فریز می‌کند).

### *ب) زمان‌بندی پس‌گرفتنی (Preemptive)*

- در شرایط **۲ و ۳** (علاوه بر ۱ و ۴) رخ می‌دهد.

- **ویژگی:** سیستم‌عامل می‌تواند پردازنده را به زور از پردازه بگیرد (مثلاً وقتی مهلت زمانی تمام شد).
- **چالش‌ها:**
  1. **شرایط مسابقه (Race Condition):** اگر دو پردازه داده مشترک داشته باشند و وسط تغییر داده توسط اولی، CPU به دومی داده شود، داده‌ها ناسازگار می‌شوند.
  2. **مشکلات هسته (Kernel):** اگر پردازه در حال اجرای یک فراخوانی سیستمی (System Call) باشد و تغییر داده‌های هسته را انجام دهد، پس‌گرفتن CPU خطرناک است. (راه‌حل: استفاده از Mutex Locks).

## ۵. اعزام‌کننده (Dispatcher)

دیسپچر ماژولی است که کنترل CPU را **عملاً** به پردازه انتخاب شده توسط Scheduler می‌دهد.
**وظایف اعزام‌کننده:**

1. تعویض متن (Context Switch): ذخیره PCB قدیم و بازیابی PCB جدید.
2. تغییر به حالت کاربر (Switch to User Mode).
3. پرش به آدرس دستورالعمل بعدی در برنامه جدید برای شروع اجرا.

- **تأخیر اعزام (Dispatch Latency):** مدت زمانی که طول می‌کشد دیسپچر یک پردازه را متوقف و دیگری را شروع کند. این زمان "سربار" (Overhead) سیستم است و باید حداقل باشد.
- **دستور vmstat در لینوکس:**
  - تعداد تعویض متن‌ها (Context Switches) را نشان می‌دهد.
  - خروجی شامل voluntary_ctxt_switches (داوطلبانه: مثلا برای I/O) و nonvoluntary (اجباری: مثلا تمام شدن کوانتوم زمانی) است.

## ۶. معیارهای زمان‌بندی (Scheduling Criteria)

برای مقایسه الگوریتم‌ها از ۵ معیار زیر استفاده می‌کنیم:

### *۱. بهره‌وری پردازنده (CPU Utilization)*

- **تعریف:** درصدی از زمان که CPU مشغول کار مفید است.
- **هدف:** ماکزیمم کردن (در سیستم‌های واقعی بین ۴۰٪ تا ۹۰٪).

### *۲. بازده (Throughput)*

- **تعریف:** تعداد پردازه‌های تکمیل شده در واحد زمان.
- **هدف:** ماکزیمم کردن.

### *۳. زمان گردش (Turnaround Time) - ⋆ بسیار مهم*

- **تعریف:** فاصله زمانی بین **ارسال** (Submit) پردازه تا **تکمیل** (Terminate) آن.
- **فرمول:** Turnaround Time = Completion Time - Arrival Time
- **شامل:** زمان انتظار در صف + زمان اجرا در CPU + زمان I/O.
- **هدف:** مینیمم کردن.

### *۴. زمان انتظار (Waiting Time) - ⋆ معیار اصلی مقایسه*

- **تعریف:** مجموع مدت زمانی که پردازه در **صف آماده (Ready Queue)** سپری کرده است.
- **نکته:** الگوریتم زمان‌بندی روی زمان اجرا یا I/O تاثیری ندارد، فقط زمان انتظار در صف را کم و زیاد می‌کند.
- **هدف:** مینیمم کردن.

### *۵. زمان پاسخ (Response Time)*

- **تعریف:** زمان سپری شده از لحظه ارسال درخواست تا تولید **اولین پاسخ** (نه تکمیل خروجی).
- **کاربرد:** در سیستم‌های **تعاملی (Interactive)** مثل دسکتاپ، این معیار از زمان گردش مهم‌تر است. واریانس (تغییرات) زمان پاسخ باید کم باشد (پیش‌بینی‌پذیر باشد).
- **هدف:** مینیمم کردن.

## شروع الگوریتم‌ها

**FCFS (First-Come, First-Served):** الگوریتم اولین الگوریتم که در انتهای فایل به آن اشاره شد: "اولین ورود، اولین سرویس". ساده‌ترین روش که با صف FIFO پیاده‌سازی می‌شود (توضیحات تکمیلی در بخش بعد).

*پایان بخش ۱ و ۲ از فصل ۵*